

Intermediary Report: Comparative Analysis of Adiabatic Error Bounds in Quantum Systems Through a Modified Landau-Zener Model

GitHub Repository

1 AIM

This project aims to provide a Python workflow for computing and comparing quantitative adiabatic error bounds for a time-dependent Hamiltonian, the sharpness of theoretical error upper bounds against the simulated error, and subsequently expand the framework to discretized adiabatic dynamics.

2 METHODOLOGY

The workflow begins by defining the Hamiltonian model and simulating its time evolution using the **QuTiP Python package**. At each temporal step t , the code extracts instantaneous eigenstates to compute the measured adiabatic error and the theoretical upper bound defined in Section 2.3.

To ensure mathematical rigor in the implementation of the bound, the operator norm $\|\cdot\|$ is calculated as the spectral norm (the largest singular value) via `numpy.linalg.norm(ord=2)`, specifically because QuTiP `H.norm()` defaults to the Frobenius norm. This evaluation is performed iteratively within the simulation loop at each timestep t , which is subsequently visualized using the `Matplotlib` package.

2.1 HAMILTONIAN DESIGN

I modeled the time-dependent Hamiltonian given by $H(t) = H_{static} + f(t)H_{dynamic}$. Equivalently, in the standard adiabatic parameterization, $H(s) = (1 - f(s))H_0 + f(s)H_1$, $s = t/T \in [0, 1]$ where T is the total evolution time. At $s = 0$, $H(0) = H_0 = H_{static}$. At $s = 1$, the Hamiltonian $H(s)$ evolves into the target Hamiltonian $H(1) = H_1 = H_{static} + f(1)H_{dynamic}$. Where H_{static} represents the time-independent part of the Hamiltonian containing the initial eigen energies and $H_{dynamic}$ is turned on through the scheduling function $f(s)$, which introduces controlled time dependence. For the three-level system studied here, the Hamiltonian (in matrix form) is:

$$H(s) = \begin{pmatrix} -f(s) + a & b & 0 \\ b & f(s) - a & c \\ 0 & c & f(s) \cdot d + e \end{pmatrix} \text{ where } a, b, c, d \in \mathbb{R} \geq 0 \quad (1)$$

2.2 QuTiP IMPLEMENTATION

The simulation models a three-level system by constructing a 3×3 quantum object (`Qobj`) within the **QuTiP Python package**. The state evolution is computed via the `sesolve` solver provided by QuTiP, which works with a Hamiltonian composed of static and dynamic parts. Then the eigenvalues and eigenstates are extracted using `H.eigenstates()` method and stored in memory.

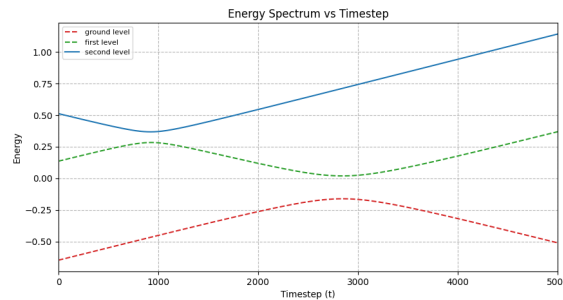


Figure 1: Plots the three eigenvalues of $H(t)$ versus time for the parameters $a = 0.5, b = 0.1, c = 0.3$, and $d = 1$. If either off-diagonal coupling b or c is set to zero, the levels cross each other. Otherwise, the coupling typically produces an avoided crossing (a nonzero minimum gap).

2.3 ANALYSIS OF ERROR BOUNDS

I study the adiabatic error between the ground state and the first excited level. To quantify this, I extract the instantaneous eigenstates at all time points and compare them with the numerically solved full time-dependent Schrödinger equation. This approach aligns with the rigorous error bounds established in *Jansen, Seiler, and Ruskai (2007)*, which analyze the adiabatic approximation for time-dependent Hamiltonian with explicit gap dependence. The objective of this comparison is to evaluate the tightness of the bound, i.e. to see how much the error bound is overestimating the actual error. The error upper bound is given by the equation,

$$A(t) \leq \frac{m\|\dot{H}(0)\|}{g(0)^2} + \frac{m\|\dot{H}(T)\|}{g(T)^2} + \int_0^T \left(\frac{m\|\ddot{H}(t)\|}{g(t)^2} + 7m\sqrt{m}\frac{\|\dot{H}(t)\|^2}{g(t)^3} \right) dt, \quad (2)$$

The integral term is evaluated numerically using `scipy.integrate.quad()` method. Because of the time dependence of the gap, it is included inside the integrand so that the bound reflects the changing gap during evolution.

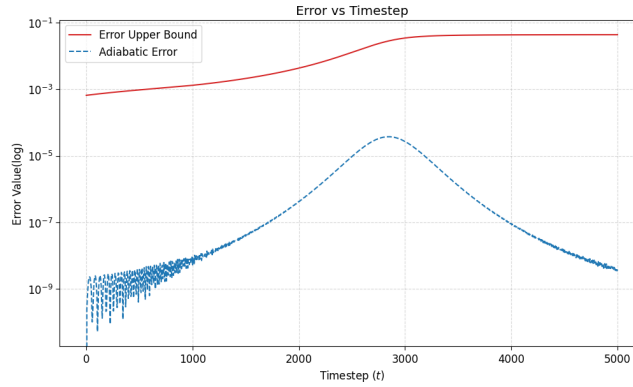


Figure 2: Compares the simulated adiabatic error with the computed upper error bound. In interpretation, the bound is meaningful only when the gap does not close during evolution i.e., there are no level crossings.

3 PROGRESS REPORT

I implemented a Python workflow using adiabatic theorem to simulate adiabatic quantum evolution for a time-dependent Hamiltonian $H(t)$ with a linear scheduling function $f(t)$. At each time step t , the code calculates the instantaneous eigenstates of the Hamiltonian and quantified the adiabatic error by comparing the numerically evolved state (from the time-dependent Schrödinger equation) with the corresponding instantaneous eigenstate. The dynamics are visualized through two plots:

- Figure 1: Instantaneous eigenvalue diagram
- Figure 2: Time-resolved plot of the adiabatic error together with the corresponding theoretical upper bound (on logarithmic scale)

4 TO BE STUDIED AND IMPLEMENTED

Having established the continuous-time framework, the focus now shifts to an investigation of the error dynamics as a function of the total evolution time T and the parameter d associated with the third eigen level 1. This analysis is motivated by the need to identify how the total duration of the process and the relative spacing of the energy levels influence the quality of the adiabatic transition.

The next phase of this project replaces the continuous-time Schrödinger evolution with a discretized model defined by a sequence of unitary (walk) operators. The resulting state is then compared against instantaneous eigenstates to determine the discrete-time adiabatic error. By implementing the corresponding unitary dynamics, this approach evaluates the impact of the discretization step size on the overall fidelity and sharpness of the upper bound.